บทที่ 2 อธิบายแบบย่อ

เอกสารนี้ไม่ใช่ส่วนของรูปเล่ม ทำไว้ให้เข้าใจว่าบทที่ 2 มีอะไรและทำไมถึงมี

บทที่ 2 ทำหน้าที่อะไรในเล่ม

เล่มปริญญานิพนธ์แบ่งหน้าที่กันชัดเจน ถ้าจำตำแหน่งของแต่ละบทได้ จะตอบคำถามได้ง่ายขึ้นมาก

บท	ตอบคำถามว่า
บทที่ 1	**ปัญหาคืออะไร** และเราจะแก้อะไรบ้าง
บทที่ 2	**ต้องรู้ทฤษฎีอะไรถึงจะแก้ปัญหานั้นได้**
บทที่ 3	**เราออกแบบระบบอย่างไร** ด้วยทฤษฎีเหล่านั้น
บทที่ 4	**ผลออกมาเป็นอย่างไร**

บทที่ 2 จึงเป็นสะพานระหว่างปัญหากับการออกแบบ ถ้ามีหัวข้อไหนที่ไม่ได้เชื่อมสองฝั่งนี้ แปลว่าหัวข้อนั้นไม่ควรอยู่

ทำไมถึงมี 9 หัวข้อใน 2.1

วิธีจำที่ง่ายที่สุดคือ **เดินตามคำสั่งซื้อหนึ่งใบ** ตั้งแต่ลูกค้าจ่ายเงินจนของออกจากคลัง แต่ละขั้นจะชนกับทฤษฎีหนึ่งข้อพอดี

ขั้นที่เกิดขึ้นจริงหน้าร้าน	ปัญหาที่ต้องแก้	หัวข้อที่รองรับ
1. พนักงานขายกดรับเงิน ระบบเขียนข้อมูลลงหลายตารางพร้อมกัน	ถ้าเขียนสำเร็จบางตาราง ข้อมูลจะเพี้ยนทันที	**2.1.2** ธุรกรรมและ ACID
2. คำสั่งซื้อเกิดขึ้นพร้อมสถานะ “รอจ่ายสินค้า”	สถานะต้องเป็นหลักฐาน ไม่ใช่ค่าที่ใครก็ตั้งได้	**2.1.4** เครื่องสถานะ
3. ระบบผลักคำสั่งซื้อไปขึ้นจอคลังทันที	คลังต้องรู้เองโดยไม่ต้องกดรีเฟรช	**2.1.6** ขับเคลื่อนด้วยเหตุการณ์
4. พนักงานคลังกดยืนยันจ่าย — ถ้าสองคนกดพร้อมกันล่ะ	ต้องกันจ่ายซ้ำและกันสต็อกเพี้ยน	**2.1.3** ล็อกเชิงบวกและเชิงลบ
5. ตัดสต็อก บันทึกการเคลื่อนไหว บันทึกว่าใครทำ	ถ้าข้อมูลไม่ตรง ต้องสืบย้อนได้ว่าเพี้ยนตอนไหน	**2.1.5** บันทึกแบบเพิ่มอย่างเดียว
6. สต็อกเหลือน้อยกว่าจุดสั่งซื้อ ระบบเตือนผู้จัดการ	ต้องเตือนก่อนของหมด ไม่ใช่รอสิ้นวัน	**2.1.9** จุดสั่งซื้อเพิ่ม
7. ผู้จัดการเปิดแดชบอร์ดดูยอด	ตัวเลขต้องตรงกับฐานข้อมูลเสมอ	**2.1.8** การรวบรวมข้อมูล
ตลอดทาง — ใครเห็นอะไรได้บ้าง	ซ่อนเมนูไม่ใช่ความปลอดภัย	**2.1.7** สิทธิ์ตามบทบาท
ทั้งหมดนี้วางอยู่บนโครงอะไร	แต่ละส่วนต้องแยกกันได้	**2.1.1** การแบ่งชั้น

ถ้าจำเรื่องเล่านี้ได้ จะไล่หัวข้อได้ครบเองโดยไม่ต้องท่อง

จัดเป็น 4 กลุ่มถ้าอยากจำแบบสั้นกว่านั้น

- **โครงระบบ** — 2.1.1
- **ทำยังไงให้ข้อมูลไม่เพี้ยน** (หัวใจของโครงงาน 4 หัวข้อ) — 2.1.2, 2.1.3, 2.1.4, 2.1.5
- **ส่งข้อมูลถึงคนที่ต้องรู้** — 2.1.6, 2.1.8
- **ใครทำอะไรได้ และเรื่องของธุรกิจ** — 2.1.7, 2.1.9

ถ้าอาจารย์ถามว่าหัวใจอยู่ตรงไหน ตอบว่า **กลุ่มที่สอง** และถ้าให้เลือกหัวข้อเดียว ตอบว่า **2.1.3**

2.2 ทำหน้าที่อะไร

6 หัวข้อ เป็น **เครื่องมือที่ทำให้ทฤษฎีใน 2.1 เกิดขึ้นจริง** ไม่ใช่รายการของที่ลง

เครื่องมือ	ทำให้ข้อไหนเกิดจริง
Node.js + Express	2.1.6 (ถือช่องทางค้างได้หลายเครื่อง) และ 2.1.1 (middleware)
MySQL + InnoDB	2.1.2 และ 2.1.3
React + Vite + Router	2.1.6 (หน้าจอเปลี่ยนเองเมื่อมีข้อมูลเข้า)
WebSocket + SSE	2.1.6
เครื่องพิมพ์ + ESC/POS	ส่วนที่คุยกับอุปกรณ์จริง
QR พร้อมเพย์	ส่วนที่รับเงิน

จุดที่ต้องเน้นตอนพูด — ไม่ได้เลือกแค่ “MySQL” แต่เลือก **InnoDB** เพราะเอนจินอื่นไม่มีธุรกรรมและล็อกทั้งตาราง ถ้าใช้ตัวนั้น 2.1.2 กับ 2.1.3 ทำไม่ได้เลย

มีรูปสัญลักษณ์ 4 รูป (รูปที่ 2.1–2.4) เฉพาะหัวข้อที่มีโลโก้ทางการ ส่วน WebSocket กับเครื่องพิมพ์ไม่มีรูป เพราะเป็นโพรโทคอลมาตรฐานและอุปกรณ์ที่ไม่ได้ระบุยี่ห้อ

2.3 ทำหน้าที่อะไร

พิสูจน์ว่า **ปัญหาที่เราแก้ ยังไม่มีใครแก้ครบในบริบทเดียวกัน** มี 6 งาน เปิดอ่านตัวจริงหมด

ผลสรุปในตารางที่ 2.1 — ทุกงานทำเรื่องสต็อก บางงานทำถึงตัดสต็อกและแจ้งเตือน **แต่ไม่มีงานไหนเลยที่มีคิวเรียลไทม์หรือกันจ่ายซ้ำ**

งานที่ใกล้ที่สุดคืองานฟิลิปปินส์ เขาเจอปัญหาเดียวกันเป๊ะ คือพนักงานหลายคนเขียนลงใบกระดาษเดียวกันแล้วข้อมูลผิด แต่เขาแก้แค่เปลี่ยนกระดาษเป็นดิจิทัล ส่วนเราแก้ที่ระดับธุรกรรมด้วยการล็อก

คำที่ต้องใช้ให้ถูก — เขียนว่า “จากการสืบค้นไม่พบ” ไม่ใช่ “ไม่มีใครทำ” เพราะเข้าไม่ถึงฐานข้อมูลที่ต้องเสียเงิน

คำถามที่น่าจะโดน

ถาม — ทำไมไม่ใช้แค่ ACID ตอบ — ACID กันธุรกรรมเดียวไม่ให้ค้างครึ่งทาง แต่ไม่ได้กันธุรกรรมสองอันทับกัน คนละเรื่อง

ถาม — ล็อกสองชั้นซ้ำซ้อนไหม ตอบ — ไม่ซ้ำ ล็อกเชิงบวกกันจ่ายซ้ำ ล็อกเชิงลบกันสต็อกเพี้ยน คนละปัญหา

ถาม — สถานะกับการล็อกเกี่ยวกันยังไง ตอบ — เป็นกลไกเดียวกัน เงื่อนไขที่ตรวจว่าสถานะเดิมต้องเป็น “รอจ่าย” ทำสองหน้าที่พร้อมกัน คือบังคับเส้นทางของเครื่องสถานะ และเป็นล็อกเชิงบวก

ถาม — ระบบรู้ไหมว่าลูกค้าจ่ายเงินแล้ว ตอบ — **ไม่รู้** พนักงานยืนยันเองจากแอปธนาคาร เป็นขอบเขตที่ประกาศไว้ตั้งแต่บทที่ 1 ตอบตรง ๆ อย่ากลบ

เตือนตัวเอง

- เอกสารอ้างอิงที่ยังไม่ได้ตรวจฉบับพิมพ์ — [1] Sommerville · [2] Bass · [8] Heizer ถ้าถูกถามเลขฉบับพิมพ์อย่าเดา
- อ้างอิงใหม่ 4 ตัวตรวจแล้วเปิดได้จริง — [6] Harel · [7] NIST SP 800-92 · [10] Sandhu · [11] Few
- หัวข้อไหนอธิบายเองไม่ได้ ให้บอกตรง ๆ ว่ายังไม่แน่น ดีกว่าเดาแล้วโดนไล่ต่อ